

EXPERIMENT-5

1. Write a Java program to demonstrate that a private member of a superclass cannot be accessed directly from a derived class.

sup.java	Output
<pre>1- public class sup { 2- private int privateVar = 10; 3- 4- public int getPrivateVar() { 5- return privateVar; 6- } 7- 8- public static void main(String[] args) { 9- sub obj = new sub(); 10- System.out.println("Private variable accessed through getter: " + obj.getPrivateVar()); 11- } 12- } 13 14- class sub extends sup { 15- public void display() { 16- // System.out.println(privateVar); // This will cause a compilation error 17- System.out.println("Private variable accessed through getter: " + getPrivateVar()); 18- } 19- }</pre>	<pre>Private variable accessed through getter: 10 === Code Execution Successful ===</pre>

2. Design a Java class Employee with attributes name, emp id, and salary. Implement a default constructor, a parameterized constructor, and methods to return the employee's name and salary. Add a method increaseSalary(double percentage) to raise the salary by a user specified percentage. Create a subclass Manager with an additional instance variable department. Develop a test program to validate these functionalities.

Employee.java

Share

Run

Output

```
1 public class Employee {
2     private String name;
3     private double salary;
4     public Employee() {
5         this.name = "";
6         this.salary = 0.0;
7     }
8     public Employee(String name, int empid, double salary) {
9         this.name = name;
10        this.salary = salary;
11    }
12    public String getName() {
13        return name;
14    }
15    public double getSalary() {
16        return salary;
17    }
18    public void increaseSalary(double percentage) {
19        if (percentage > 0) {
20            salary += salary * percentage / 100;
21        }
22    }
23 }
24 class Manager extends Employee {
25     private String department;
```

Private variable accessed through getter: 10
=== Code Execution Successful ===

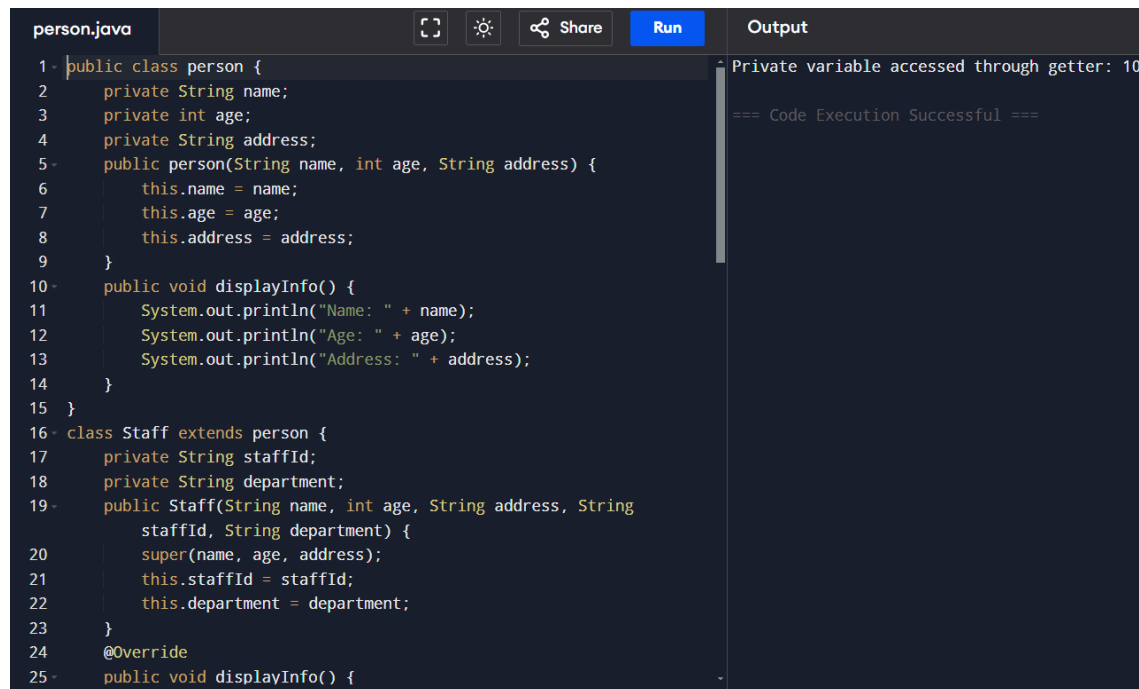
Employee.java

Share

Run

```
26     public Manager(String name, int empid, double salary, String
27         department) {
28         super(name, empid, salary);
29         this.department = department;
30     }
31     public String getDepartment() {
32         return department;
33     }
34     class TestProgram {
35     public static void main(String[] args) {
36         Employee emp = new Employee("John Doe", 12345, 50000);
37         System.out.println("Employee Name: " + emp.getName());
38         System.out.println("Employee Salary: " + emp.getSalary());
39         emp.increaseSalary(10);
40         System.out.println("Employee Salary after increase: " + emp
41             .getSalary());
42         Manager mgr = new Manager("Jane Smith", 67890, 70000, "HR");
43         System.out.println("Manager Name: " + mgr.getName());
44         System.out.println("Manager Salary: " + mgr.getSalary());
45         System.out.println("Manager Department: " + mgr
46             .getDepartment());
47         mgr.increaseSalary(15);
48         System.out.println("Manager Salary after increase: " + mgr
49             .getSalary());
50     }
51 }
```

3. A university has different types of people associated with it, including staff members and students. The base class Person contains common attributes such as name, age, and address. The class Staff extends Person and adds attributes like staffId and department. Further, a subclass Professor extends Staff by introducing an additional attribute, specialization, and a method conductLecture(). Similarly, the Student class extends Person and adds studentId and course. Finally, the subclass GraduateStudent extends Student, adding researchTopic and a method submitThesis(). Implement this university management system in Java using multilevel inheritance and method overriding. Demonstrate polymorphism by creating an array of Person objects containing instances of Professor and GraduateStudent, and call their respective methods.



```
person.java
1 public class person {
2     private String name;
3     private int age;
4     private String address;
5     public person(String name, int age, String address) {
6         this.name = name;
7         this.age = age;
8         this.address = address;
9     }
10    public void displayInfo() {
11        System.out.println("Name: " + name);
12        System.out.println("Age: " + age);
13        System.out.println("Address: " + address);
14    }
15 }
16 class Staff extends person {
17     private String staffId;
18     private String department;
19     public Staff(String name, int age, String address, String
20         staffId, String department) {
21         super(name, age, address);
22         this.staffId = staffId;
23         this.department = department;
24     }
25     @Override
26     public void displayInfo() {
```

Output

Private variable accessed through getter: 10

=== Code Execution Successful ===

```
person.java  [ ] [ ] [ ] Share Run
26      super.displayInfo();
27      System.out.println("Staff ID: " + staffId);
28      System.out.println("Department: " + department);
29  }
30  }
31  class Professor extends Staff {
32      private String specialization;
33      public Professor(String name, int age, String address, String
        staffId, String department, String specialization) {
34          super(name, age, address, staffId, department);
35          this.specialization = specialization;
36      }
37      public void conductLecture() {
38          System.out.println("Conducting lecture on " + specialization
        );
39      }
40      @Override
41      public void displayInfo() {
42          super.displayInfo();
43          System.out.println("Specialization: " + specialization);
44      }
45  }
46  class Student extends person {
47      private String studentId;
48      private String course;
49      public Student(String name, int age, String address, String
```

```
person.java  [ ] [ ] [ ] Share Run
50      super(name, age, address);
51      this.studentId = studentId;
52      this.course = course;
53  }
54  @Override
55  public void displayInfo() {
56      super.displayInfo();
57      System.out.println("Student ID: " + studentId);
58      System.out.println("Course: " + course);
59  }
60  }
61  class GraduateStudent extends Student {
62      private String researchTopic;
63      public GraduateStudent(String name, int age, String address,
        String studentId, String course, String researchTopic) {
64          super(name, age, address, studentId, course);
65          this.researchTopic = researchTopic;
66      }
67      public void submitThesis() {
68          System.out.println("Submitting thesis on " + researchTopic);
69      }
70      @Override
71      public void displayInfo() {
72          super.displayInfo();
73          System.out.println("Research Topic: " + researchTopic);
74      }
```

4. Design an abstract class named `Vehicle` with abstract methods such as `startEngine()` and `stopEngine()`, as well as a non-abstract method `serviceInfo()` that prints basic servicing instructions. Include a couple of protected or private fields (e.g., `make`, `model`) and a constructor for `Vehicle` that initializes those fields. Create a concrete class `Car` that extends `Vehicle` and provides implementations for `startEngine()` and `stopEngine()`. Within these methods, include print statements or logic that simulates starting and stopping a car engine. Finally, instantiate a `Car` object in the main method, call all available methods (including the inherited non-abstract method), and verify that your abstraction works as intended.

```
1  public abstract class Vehicle {
2      protected String make;
3      protected String model;
4
5      public Vehicle(String make, String model) {
6          this.make = make;
7          this.model = model;
8      }
9
10     public abstract void startEngine();
11
12     public abstract void stopEngine();
13
14     public void serviceInfo() {
15         System.out.println("Basic servicing instructions for " + make + " " + model + ":");
16         System.out.println(x:"1. Check engine oil level.");
17         System.out.println(x:"2. Inspect tire pressure.");
18         System.out.println(x:"3. Replace air filter as needed.");
19     }
20 }
```

```
J Car.java 5 ● J Vehicle.java 1
C: > Users > aasth > AppData > Local > Temp > a9cec695-b6d7-4f6a-a24c-d242cb609
1 public class Car extends Vehicle {
2
3     public Car(String make, String model) {
4         super(make, model);
5     }
6     @Override
7     public void startEngine() {
8         System.out.println(x:"Car engine starting");
9     }
10    @Override
11    public void stopEngine() {}
12    System.out.println(x:"Car engine stopping");
13 }
14
15 Run | Debug
16 public static void main(String[] args) {
17     Car myCar = new Car(make:"Toyota", model:"Camry");
18     Run main | Debug main
19     myCar.startEngine();
20     myCar.stopEngine();
21     myCar.serviceInfo();
22 }
23 }

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\aasth> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-
dt_ws\jdt.ls-java-project\bin' 'Car'
Car engine starting
Car engine stopping
Basic servicing instructions for Toyota Camry:
1. Check engine oil level.
2. Inspect tire pressure.
3. Replace air filter as needed.
```

5. Imagine you're designing a restaurant management system where you have a base class called Chef with a method makeSpecialDish(). This method simply prints a generic message indicating that the chef is making a special dish. Next, create three subclasses called ItalianChef, ChineseChef, and MexicanChef—each overrides makeSpecialDish() with a

distinct print statement, such as "Making pasta", "Making dumplings", or "Making tacos".

In your main method, declare an array or list of Chef references, but assign each element to a different subclass (ItalianChef, ChineseChef, MexicanChef). Use a loop to call the makeSpecialDish() method on each array element and observe how runtime polymorphism ensures that the correct subclass version of makeSpecialDish() is invoked.

Chef.java	   Share  Run	Output
<pre>1 public class Chef { 2 public void makeSpecialDish() { 3 System.out.println("The chef is making a special dish."); 4 } 5 6 public static void main(String[] args) { 7 Chef[] chefs = {new ItalianChef(), new ChineseChef(), new 8 MexicanChef()}; 9 10 for (Chef chef : chefs) { 11 chef.makeSpecialDish(); 12 } 13 } 14 class ItalianChef extends Chef { 15 @Override 16 public void makeSpecialDish() { 17 System.out.println("Making pasta"); 18 } 19 } 20 class ChineseChef extends Chef { 21 @Override 22 public void makeSpecialDish() { 23 System.out.println("Making dumplings"); 24 } 25 }</pre>	<pre>Making pasta Making dumplings Making tacos === Code Execution Succ</pre>	

```
25 }
26 class MexicanChef extends Chef {
27     @Override
28     public void makeSpecialDish() {
29         System.out.println("Making tacos");
30     }
31 }
```